JSC «International Information Technology University»

Faculty of Business, Media and Management

Department: Digital and Intelligent Systems

Discipline: RWPSF 3305 – Web Application Development with Spring Framework

Laboratory Work 8. The data access layer

Connecting PostgreSQL, entities and associations (@OneToMany, @ManyToMany) · Spring Data JPA repositories, derived and JPQL queries · pagination and sorting

Student name: Kuat Bektursyn

Group: IT1-2409IS

Programme: 6B06101 Information Systems, Year 3, Semester 5

Date of work: 26 October 2026

Lecturer: Yemberdiyeva Aknur, Senior Lector

Individual variant: No. 7 — Window and keyset scrolling

Repository: github.com/AkAyu44/library-portal, branch lab08

Almaty, 2026

1. Aim of the work and repository

To replace the in-memory list that has carried the catalogue since week 1 with a real database. By the end of the work the project stores books in PostgreSQL, the model has three entities joined by a one-to-many and a many-to-many association, the repository answers five derived queries and two written in JPQL, and the REST API of laboratory work no. 7 returns a page instead of a table.

Repository, branch lab08: `https://github.com/AkAyu44/library-portal/tree/lab08`

Commit of the work: `https://github.com/AkAyu44/library-portal/commit/dc290ed` — "lab08: PostgreSQL, entities, associations, queries, paging", 95 files changed.

Project: the work is done on top of the same project as labs 1–7 — *Library Portal* (Spring Boot 4.1.1, Java 25); branch lab08 was created from main. Nothing was rebuilt from scratch: the REST API, DTOs and ProblemDetail of lab 7, the aspects of lab 4 and the forms of lab 6 keep working; only the storage under BookService changed.

Environment and deviations from the handout

Table 1. *Workstation*

Component	Required	Used in this work
JDK	Java 25 LTS	java version "25.0.2" 2026-01-20 LTS
Project from lab 7	spring-lab-01, branch main	library-portal (the course project with labs 1–7), branch main → lab08
PostgreSQL	16 or newer, or Docker	PostgreSQL 16.14 (official EDB binaries, local server on :5432); Docker is not installed
Database and user	springlab / springlab	springlab / springlab; plus springlab_test for the tests
Spring	—	Spring Boot 4.1.1, Spring Data JPA 4.1.1, Hibernate ORM 7.4.5.Final, PostgreSQL JDBC 42.7.13
Git	2.40+	branch lab08 pushed to origin

Deviations: (1) the project is library-portal rather than spring-lab-01 — it is the same course project that already contains labs 1–7; (2) Book keeps the fields of labs 6–7 (available, publishedOn) next to the fields of the handout; (3) the API and the form still send the author as a name, so the service finds or creates the Author entity; (4) besides the console log, a small Hibernate StatementInspector (persistence/SqlRecorder) counts the SQL statements, so the numbers in the tables of tasks 3 and 4 are measured, not counted by eye.

2. Task 1. PostgreSQL and the first entity

1.1. Branch and database

```
PowerShell — library-portal (lab08)
$ git checkout main && git pull
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
$ git checkout -b lab08
Switched to a new branch 'lab08'
$ psql --version
psql (PostgreSQL) 16.14
$ psql -U postgres -c "create user springlab with password 'springlab';"
CREATE ROLE
$ psql -U postgres -c "create database springlab owner springlab;"
CREATE DATABASE
$ psql -U postgres -c "\l springlab"
                                              List of databases
   Name    |   Owner   | Encoding | Locale Provider | Collate | Ctype | ICU Locale | ICU Rules | Access privileges
-----------+-----------+----------+-----------------+---------+-------+------------+-----------+-------------------
 springlab | springlab | UTF8     | libc            | C       | C     |            |           |
(1 row)
```

Figure 1. *Branch lab08 created from main; user and database springlab created in PostgreSQL 16.14*

1.2. Dependencies

Listing 1. *pom.xml — the starter and the driver; Spring Boot manages both versions*

```
        <!-- Lab 8: the data access layer — Spring Data JPA (Hibernate) and the PostgreSQL driver;
             Spring Boot manages both versions -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
```

1.3. The datasource

Listing 2. *application.yml — datasource and JPA settings*

```
# Lab 8: PostgreSQL instead of the in-memory list
datasource:
  url: jdbc:postgresql://localhost:5432/springlab
  username: springlab
  password: springlab
jpa:
  open-in-view: false          # no lazy loading during view rendering: the service is the boundary
  hibernate:
    ddl-auto: create-drop      # for this class only: schema from the entities, dropped on shutdown
  show-sql: true               # you will need to read the SQL
  properties:
    hibernate.format_sql: true
data:
  web:
```

```
      pageable:
        max-page-size: 50
```

open-in-view: false closes the persistence context when the service transaction ends, so nothing is loaded lazily while JSON or Thymeleaf is being rendered. ddl-auto: create-drop builds the schema from the entities at start-up and drops it on shutdown — for this class only; application-prod.yml sets ddl-auto: validate.

1.4. Book is an entity

Listing 3. *domain/Book.java — the entity (the record of labs 5–7 became a class)*

```
import java.time.Instant;
import java.time.LocalDate;
import java.util.Collections;
import java.util.HashSet;
import java.util.Set;

/**
 * Lab 5: the domain object of the catalogue (id, title, author, year).
 * Lab 6: genre, availability and the publication date (variant 7).
 * Lab 7: createdAt — set by the server, never sent by a client.
 * Lab 8: no longer a record but a JPA entity mapped to the table "book".
 * JPA needs a no-argument constructor, a non-final class and mutable fields
 * (the persistence context writes them after loading and tracks their changes),
 * so a record — final, with final fields and no no-arg constructor — cannot be an entity.
 * Book is the owning side of both associations: it holds author_id and declares book_tag.
 */
@Entity
@Table(name = "book")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)   // PostgreSQL: bigint generated by default as
identity
    private Long id;

    @Column(nullable = false, length = 120)
    private String title;

    @Enumerated(EnumType.STRING)                          // 'PROGRAMMING', not 2 — safe when the enum is
reordered
    @Column(nullable = false, length = 20)
    private Genre genre;

    private int year;

    private boolean archived;

    // Lab 6: kept from the form of the catalogue
    private boolean available = true;

    @Column(name = "published_on")
    private LocalDate publishedOn;

    @Column(name = "created_at", updatable = false)
    private Instant createdAt = Instant.now();

    @Version                                              // optimistic locking: where id = ? and version = ?
    private long version;

    // the owning side of book -> author: the foreign key author_id lives in "book"
    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "author_id")
    private Author author;

    // the owning side of book <-> tag: it declares the join table book_tag(book_id, tag_id)
    @ManyToMany
    @JoinTable(name = "book_tag",
               joinColumns = @JoinColumn(name = "book_id"),
               inverseJoinColumns = @JoinColumn(name = "tag_id"))
    @BatchSize(size = 20)                                // tags of a whole page in one select ... where
book_id in (...)
    private Set<Tag> tags = new HashSet<>();

    protected Book() { }                                 // JPA needs this one; protected keeps it out of our
code

    public Book(String title, Genre genre, int year) {
        this.title = title;
        this.genre = genre;
        this.year = year;
    }

    public void addTag(Tag tag) {
        tags.add(tag);
    }

    public void clearTags() {
        tags.clear();
    }

    public Long getId() { return id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public Genre getGenre() { return genre; }
    public void setGenre(Genre genre) { this.genre = genre; }

    public int getYear() { return year; }
    public void setYear(int year) { this.year = year; }

    public boolean isArchived() { return archived; }
    public void setArchived(boolean archived) { this.archived = archived; }

    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }

    public LocalDate getPublishedOn() { return publishedOn; }
    public void setPublishedOn(LocalDate publishedOn) { this.publishedOn = publishedOn; }

    public Instant getCreatedAt() { return createdAt; }

    public long getVersion() { return version; }

    public Author getAuthor() { return author; }

    /** Called by Author.addBook — the one place that keeps both sides of the link in step. */
    void setAuthor(Author author) { this.author = author; }

    public Set<Tag> getTags() { return Collections.unmodifiableSet(tags); }

    /** The associations are left out: printing them would load them, or recurse through Author.books. */
    @Override
    public String toString() {
        return "Book{id=" + id + ", title='" + title + "', genre=" + genre + ", year=" + year + "}";
```

```
    }
}
```

Why it is no longer a record: JPA needs a no-argument constructor, a non-final class (Hibernate creates proxy subclasses for lazy loading) and non-final fields that it can set after reading a row and later compare for dirty checking. A record is final, its fields are final and it has only the canonical constructor. The protected constructor exists for JPA only; our code uses Book(title, genre, year).

1.5. The repository replaces the in-memory list; the application starts against PostgreSQL

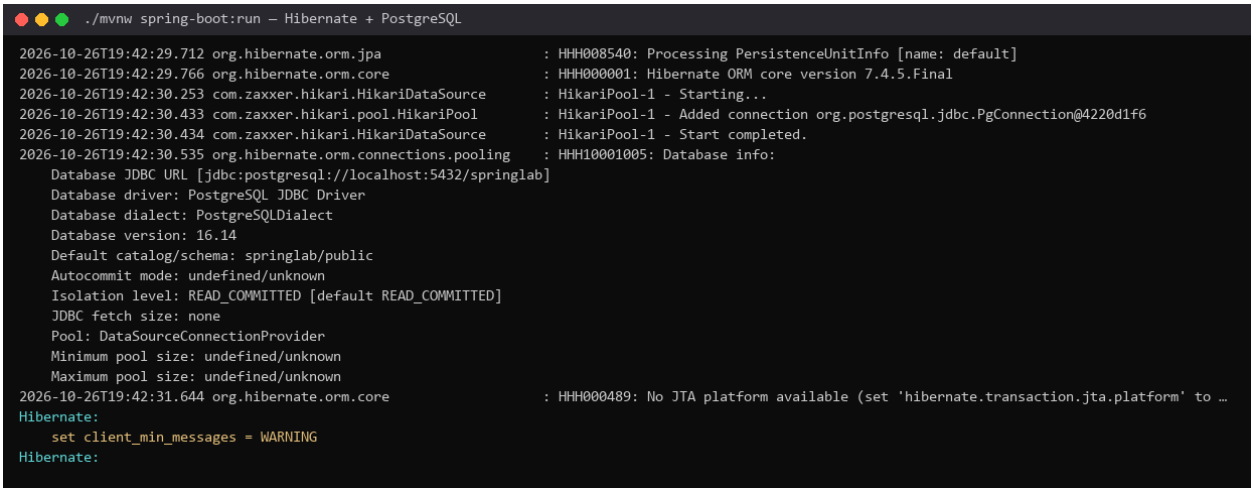


Figure 2. *Start-up: Hibernate 7.4.5 connects to jdbc:postgresql://localhost:5432/springlab, PostgreSQL 16.14, PostgreSQLDialect — not H2, and no in-memory list is left*

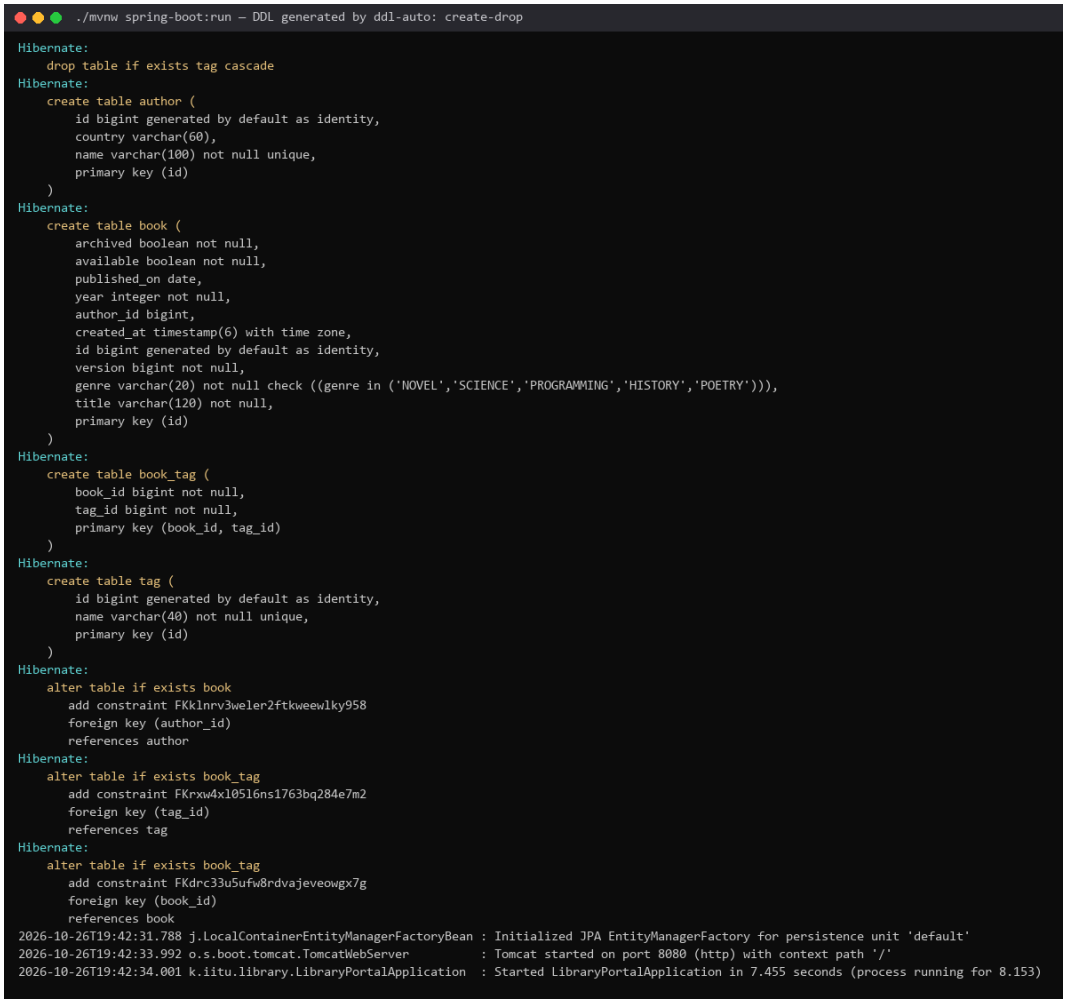


Figure 3. *The DDL printed by show-sql: tables author, book, book_tag, tag and their foreign keys*

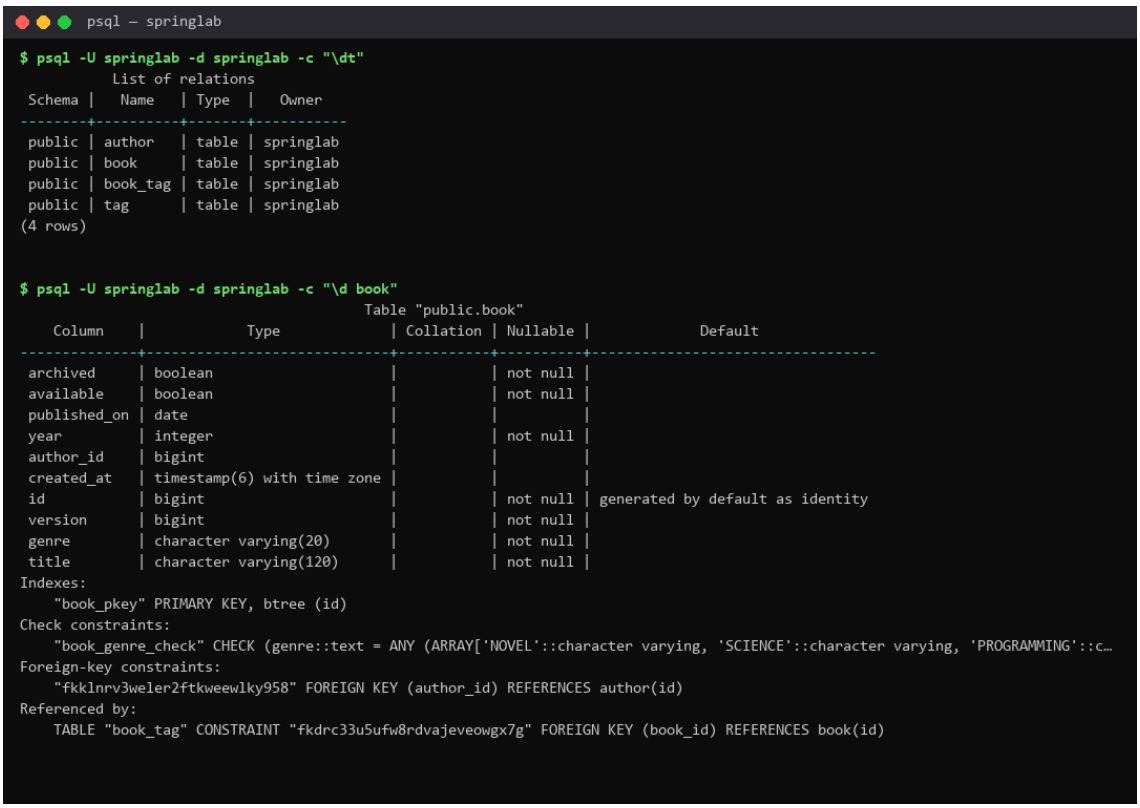


Figure 4. *psql: \dt and \d book — the schema as the database reports it*

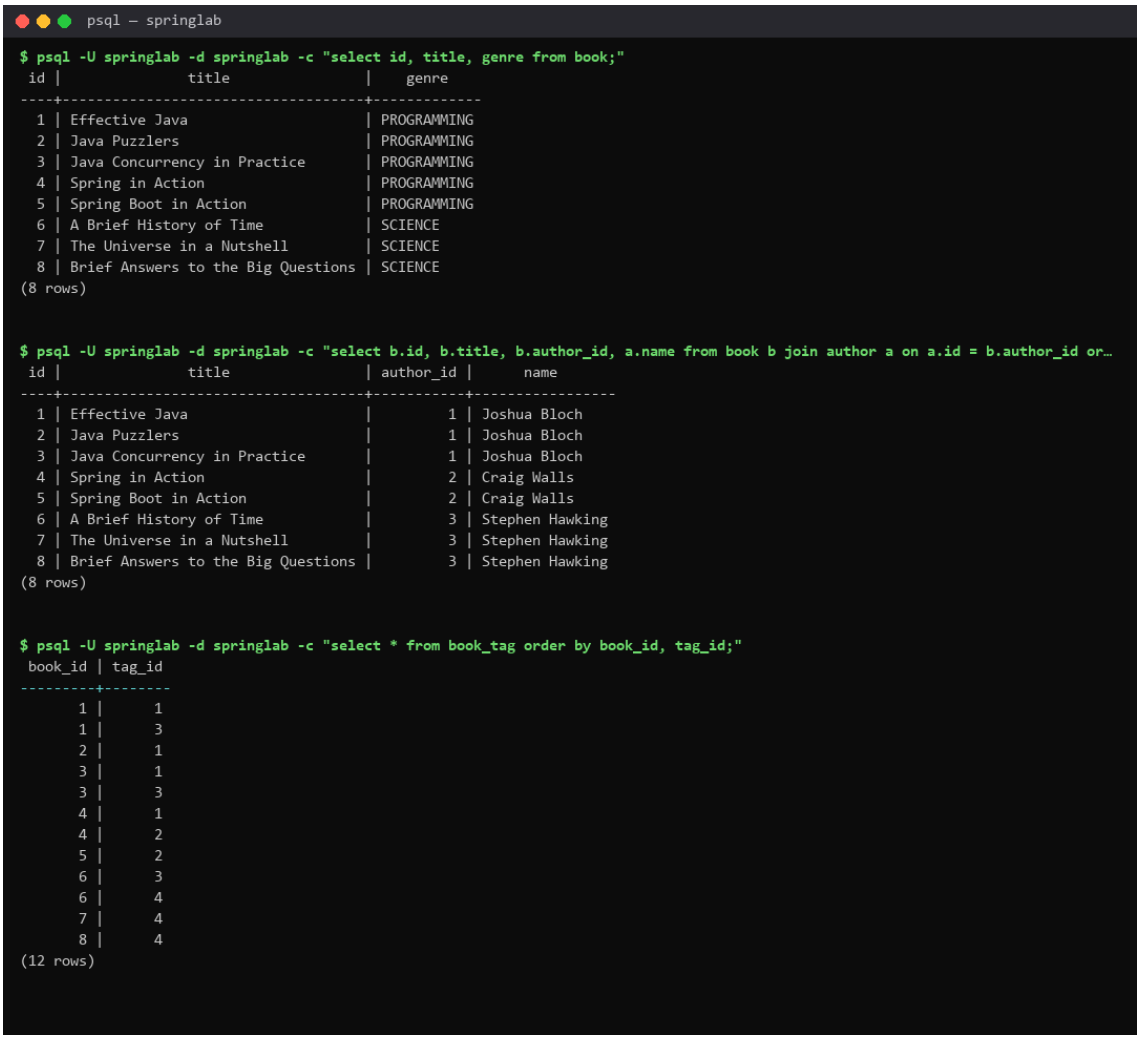


Figure 5. *psql: select id, title, genre from book; author_id of every book; the rows of book_tag*

curl — page 0, size 3

```
$ curl -s "localhost:8080/api/books?page=0&size=3"
{
    "content": [
        {
            "id": 6,
            "title": "A Brief History of Time",
            "author": "Stephen Hawking",
            "year": 1988,
            "genre": "SCIENCE",
            "available": true,
            "archived": false,
            "publishedOn": "1988-04-01",
            "tags": [
                "classic",
                "physics"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        },
        {
            "id": 8,
            "title": "Brief Answers to the Big Questions",
            "author": "Stephen Hawking",
            "year": 2018,
            "genre": "SCIENCE",
            "available": true,
            "archived": false,
            "publishedOn": "2018-10-16",
            "tags": [
                "physics"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        },
        {
            "id": 1,
            "title": "Effective Java",
            "author": "Joshua Bloch",
            "year": 2018,
            "genre": "PROGRAMMING",
            "available": true,
            "archived": false,
            "publishedOn": "2018-01-06",
            "tags": [
                "classic",
                "java"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        }
    ],
    "page": {
        "size": 3,
        "number": 0,
        "totalElements": 8,
        "totalPages": 3
    }
}
```

Figure 6. *curl localhost:8080/api/books?page=0&size=3 still answers — now from the database*

3. Task 2. Associations

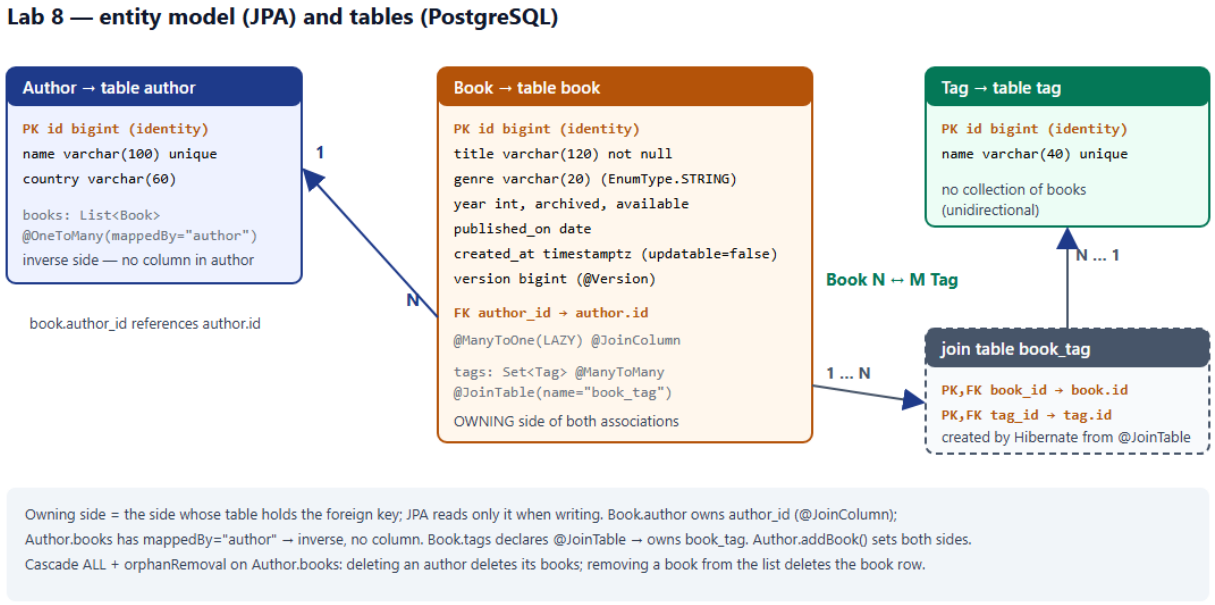


Figure 7. *Entity diagram: Author 1—N Book, Book N—M Tag, with cardinalities and owning sides*

Written down before running anything: the author foreign key will be in table book (column author_id), because Book.author is @ManyToOne with @JoinColumn — the owning side; Author.books carries mappedBy and therefore no column; the many-to-many is stored in a third table book_tag(book_id, tag_id) declared by @JoinTable on Book.tags. The schema in figure 3, figure 4 and the psql output below confirm it.

Listing 4. *Book — the owning side of both associations*

```
    // the owning side of book -> author: the foreign key author_id lives in "book"
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;

    // the owning side of book <-> tag: it declares the join table book_tag(book_id, tag_id)
    @ManyToMany
    @JoinTable(name = "book_tag",
               joinColumns = @JoinColumn(name = "book_id"),
               inverseJoinColumns = @JoinColumn(name = "tag_id"))
    @BatchSize(size = 20)                                  // tags of a whole page in one select ...
where book_id in (...)
    private Set<Tag> tags = new HashSet<>();
```

Listing 5. *Author — the inverse side and the helper that keeps both sides in step*

```
    // ALL: saving or deleting an author saves or deletes its books;
    // orphanRemoval: a book taken out of this list is deleted, not left with author_id = null
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();

    protected Author() { }

    public Author(String name, String country) {
        this.name = name;
        this.country = country;
    }
```

```
    /** Keeps both sides in step: JPA reads only the owning side (Book.author) when it writes author_id.
*/
    public void addBook(Book book) {
        books.add(book);
        book.setAuthor(this);
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.setAuthor(null);
    }
```

2.2. Seed data

Listing 6. *persistence/DataSeeder.java — a CommandLineRunner for the development profiles*

```
    @Override
    @Transactional
    public void run(String... args) {
        if (authors.count() > 0) {
            return;
        }
        Map<String, Tag> tag = Map.of(
                "java", tags.save(new Tag("java")),
                "spring", tags.save(new Tag("spring")),
                "classic", tags.save(new Tag("classic")),
                "physics", tags.save(new Tag("physics")));

        Author bloch = new Author("Joshua Bloch", "USA");
        bloch.addBook(book("Effective Java", Genre.PROGRAMMING, 2018, LocalDate.of(2018, 1, 6), true,
                tag.get("java"), tag.get("classic")));
        bloch.addBook(book("Java Puzzlers", Genre.PROGRAMMING, 2005, LocalDate.of(2005, 7, 1), true,
                tag.get("java")));
        bloch.addBook(book("Java Concurrency in Practice", Genre.PROGRAMMING, 2006, LocalDate.of(2006, 5, 9),
false,
                tag.get("java"), tag.get("classic")));

        Author walls = new Author("Craig Walls", "USA");
        walls.addBook(book("Spring in Action", Genre.PROGRAMMING, 2022, LocalDate.of(2022, 2, 1), false,
                tag.get("java"), tag.get("spring")));
        walls.addBook(book("Spring Boot in Action", Genre.PROGRAMMING, 2015, LocalDate.of(2015, 12, 1), true,
                tag.get("spring")));

        Author hawking = new Author("Stephen Hawking", "United Kingdom");
        hawking.addBook(book("A Brief History of Time", Genre.SCIENCE, 1988, LocalDate.of(1988, 4, 1), true,
                tag.get("physics"), tag.get("classic")));
        hawking.addBook(book("The Universe in a Nutshell", Genre.SCIENCE, 2001, LocalDate.of(2001, 11, 6),
true,
                tag.get("physics")));
        hawking.addBook(book("Brief Answers to the Big Questions", Genre.SCIENCE, 2018, LocalDate.of(2018, 10,
16), true,
                tag.get("physics")));

        authors.save(bloch);      // cascade = ALL: the INSERTs of the three books follow the author's
        authors.save(walls);
        authors.save(hawking);
        log.info
```

Three authors, eight books, four tags. Joshua Bloch and Stephen Hawking own three books each; Effective Java, Java Concurrency in Practice, Spring in Action and A Brief History of Time carry two tags each. The runner is

active in the profiles "default" (a run without a profile) and "dev"; prod never gets test data. The run issued 27 INSERT statements: 4 tag + 3 author + 8 book (cascade = ALL from the author) + 12 book_tag (written at commit).

CommandLineRunner DataSeeder — INSERTs (fragment)

```
Hibernate:
    select
        count(*)
    from
        author a1_0
Hibernate:
    insert
    into
        tag
        (name)
    values
        (?)
Hibernate:
    insert
    into
        tag
        (name)
    values
        (?)
Hibernate:
    insert
    into
        tag
        (name)
    values
        (?)
Hibernate:
    insert
    into
        tag
        (name)
    values
        (?)
Hibernate:
    insert
    into
        tag
        (name)
    values
        (?)
Hibernate:
    insert
    into
        author
        (country, name)
    values
        (?, ?)
Hibernate:
    insert
    into
        book
        (archived, author_id, available, created_at, genre, published_on, title, version, year)
    values
        (?, ?, ?, ?, ?, ?, ?, ?, ?)
Hibernate:
    insert
    into
        book
        (archived, author_id, available, created_at, genre, published_on, title, version, year)
    values
        (?, ?, ?, ?, ?, ?, ?, ?, ?)
Hibernate:
    insert
    into
        book
        (archived, author_id, available, created_at, genre, published_on, title, version, year)
    values
```

Figure 8. *The INSERTs of the seed: every book is inserted with author_id already set*

2.3. What the database shows

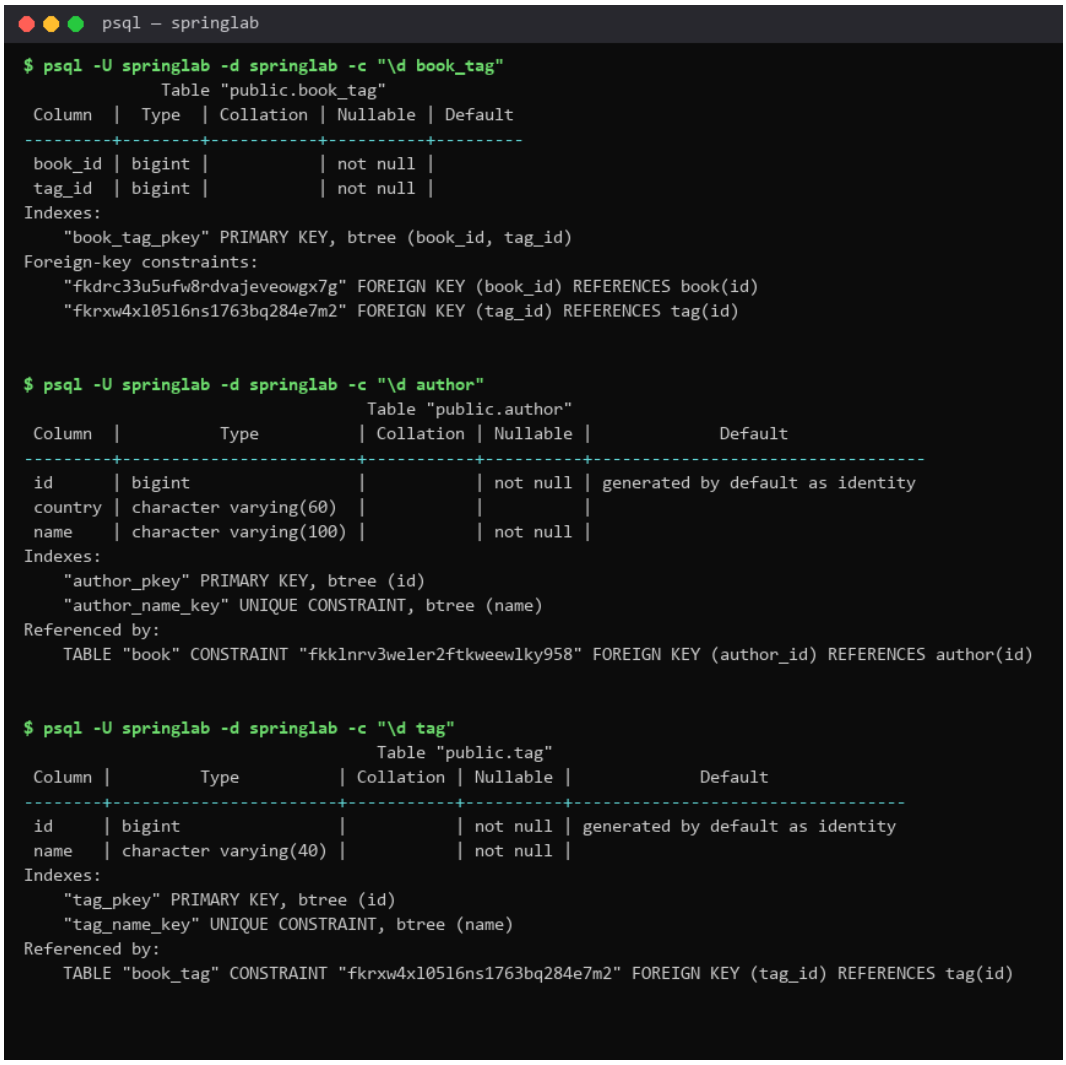


Figure 9. *psql: \d book_tag, \d author, \d tag*

Table 2. *Task 2.3 — filled in from what the database shows*

Question	Your answer	How you checked it
Which tables were created?	author, book, book_tag, tag	\dt (figure 4) and the create table statements (figure 3)
Where is the author foreign key?	book.author_id bigint → author(id), constraint fkklnrv3weler2ftkweewlky958	\d book, section "Foreign-key constraints"
What columns does book_tag have?	book_id bigint not null, tag_id bigint not null; primary key (book_id, tag_id); both are foreign keys	\d book_tag (figure 9)
Does author have a books column?	No — only id, country, name. Author.books is mappedBy = "author", the inverse side	\d author (figure 9)

The two mistakes from the handout were checked against the SQL. Without mappedBy JPA would treat Author.books as a second owning side and create a third table author_books. Adding a book only with author.getBooks().add(book) would insert author_id = null, because JPA reads only the owning side. Author.addBook() sets both sides — which is why the inserts in figure 8 already carry author_id.

4. Task 3. Queries

Listing 7. *repository/BookRepository.java — derived queries, JPQL, bulk update, N+1 cure, paging, Window*

```
import java.util.List;
import java.util.Optional;

/**
 * Lab 5 kept the catalogue in a ConcurrentHashMap; Lab 8 replaces it with an interface that
 * nobody implements by hand. At startup Spring Data creates a proxy (SimpleJpaRepository for the
 * CRUD methods of JpaRepository) and parses every other method name into JPQL.
 */
public interface BookRepository extends JpaRepository<Book, Long> {

    // ---------------- Task 3.1: five derived queries ----------------

    /** Also the declarative cure for N+1: the author comes in the same select (left join). */
    @EntityGraph(attributePaths = "author")
    List<Book> findByGenre(Genre genre);

    /** where upper(b.title) like upper(?) escape '\' — the argument is wrapped in % ... %. */
    List<Book> findByTitleContainingIgnoreCase(String fragment);

    List<Book> findByYearBetweenOrderByYearDesc(int from, int to);

    /** Subject "findTop3", predicate "ByGenre", order "OrderByYearDesc" → fetch first 3 rows only. */
    List<Book> findTop3ByGenreOrderByYearDesc(Genre genre);

    /** A nested property: b.author.name → an inner join to "author". */
    long countByAuthorName(String name);

    // used by BookService for the Lab 7 duplicate rule (409) — no more scanning a list in Java
    boolean existsByTitleIgnoreCaseAndAuthorNameIgnoreCase(String title, String author);

    boolean existsByTitleIgnoreCaseAndAuthorNameIgnoreCaseAndIdNot(String title, String author, Long id);

    // ---------------- Task 3.2: JPQL and a bulk update ----------------

    @Query("""
           select b from Book b
           where b.genre = :genre and b.year >= :year
           order by b.title""")
    List<Book> search(@Param("genre") Genre genre, @Param("year") int year);

    /** One UPDATE statement, the persistence context is bypassed; @Transactional is on the caller. */
    @Modifying(clearAutomatically = true)
    @Query("update Book b set b.archived = true where b.year < :year")
    int archiveOlderThan(@Param("year") int year);

    /** Lab 4 statistics, now counted by the database: one row per genre. */
    @Query("select b.genre, count(b) from Book b group by b.genre")
    List<Object[]> countPerGenre();

    // ---------------- Task 3.4: the cure for N+1 ----------------

    @Query("select b from Book b join fetch b.author")
    List<Book> findAllWithAuthor();

    /** One book with everything the details page and GET /api/books/{id} print. */
    @Query("select b from Book b left join fetch b.author left join fetch b.tags where b.id = :id")
    Optional<Book> findWithDetailsById(@Param("id") Long id);

    // ---------------- Task 4: pagination and sorting ----------------

    /** JpaRepository.findAll(Pageable) re-declared only to add the entity graph. */
    @Override
    @EntityGraph(attributePaths = "author")
    Page<Book> findAll(Pageable pageable);
```

```
    @EntityGraph(attributePaths = "author")
    Page<Book> findByGenre(Genre genre, Pageable pageable);

    @EntityGraph(attributePaths = "author")
    Page<Book> findByAuthorNameContainingIgnoreCase(String author, Pageable pageable);

    @EntityGraph(attributePaths = "author")
    Page<Book> findByGenreAndAuthorNameContainingIgnoreCase(Genre genre, String author, Pageable pageable);

    /** Task 4.3: the same page as a Slice — no count query, one extra row tells whether there is more. */
    @EntityGraph(attributePaths = "author")
    Slice<Book> findSliceBy(Pageable pageable);

    // ---------------- Individual variant 7: Window and keyset scrolling ----------------

    /**
     * Keyset (seek) scrolling: the next window starts after the sort keys of the last row —
     * where (year < ? or year = ? and id < ?) — instead of skipping OFFSET rows.
     * BookService never passes ScrollPosition.keyset() (the empty initial position): Spring Data
     * caches the query rendered for the first call, and a query rendered without the keyset
     * predicate would then be reused for every later window (found in this lab, see the report).
     */
    @EntityGraph(attributePaths = "author")
    Window<Book> findBy(ScrollPosition position, Sort sort, Limit limit);
}
```

3.3. Every method called once

All methods were called through the temporary endpoint POST /api/lab8/queries (api/Lab8Controller → persistence/QueryLab). Every call runs in its own transaction and returns the rows and the exact SQL that Hibernate sent.

Table 3. *Task 3.3 — measured with POST /api/lab8/queries*

Method	Rows returned	Number of SQL statements
findByGenre(PROGRAMMING)	5	1
findByTitleContainingIgnoreCase("java")	3	1
findByYearBetweenOrderByYearDesc(2005, 2018)	5	1
findTop3ByGenreOrderByYearDesc(PROGRAMMING)	3	1
countByAuthorName("Joshua Bloch")	3	1
search(PROGRAMMING, 2010) [JPQL]	3	1
archiveOlderThan(2000) [bulk update]	1	1
findByTitleContainingIgnoreCase("%")	0	1

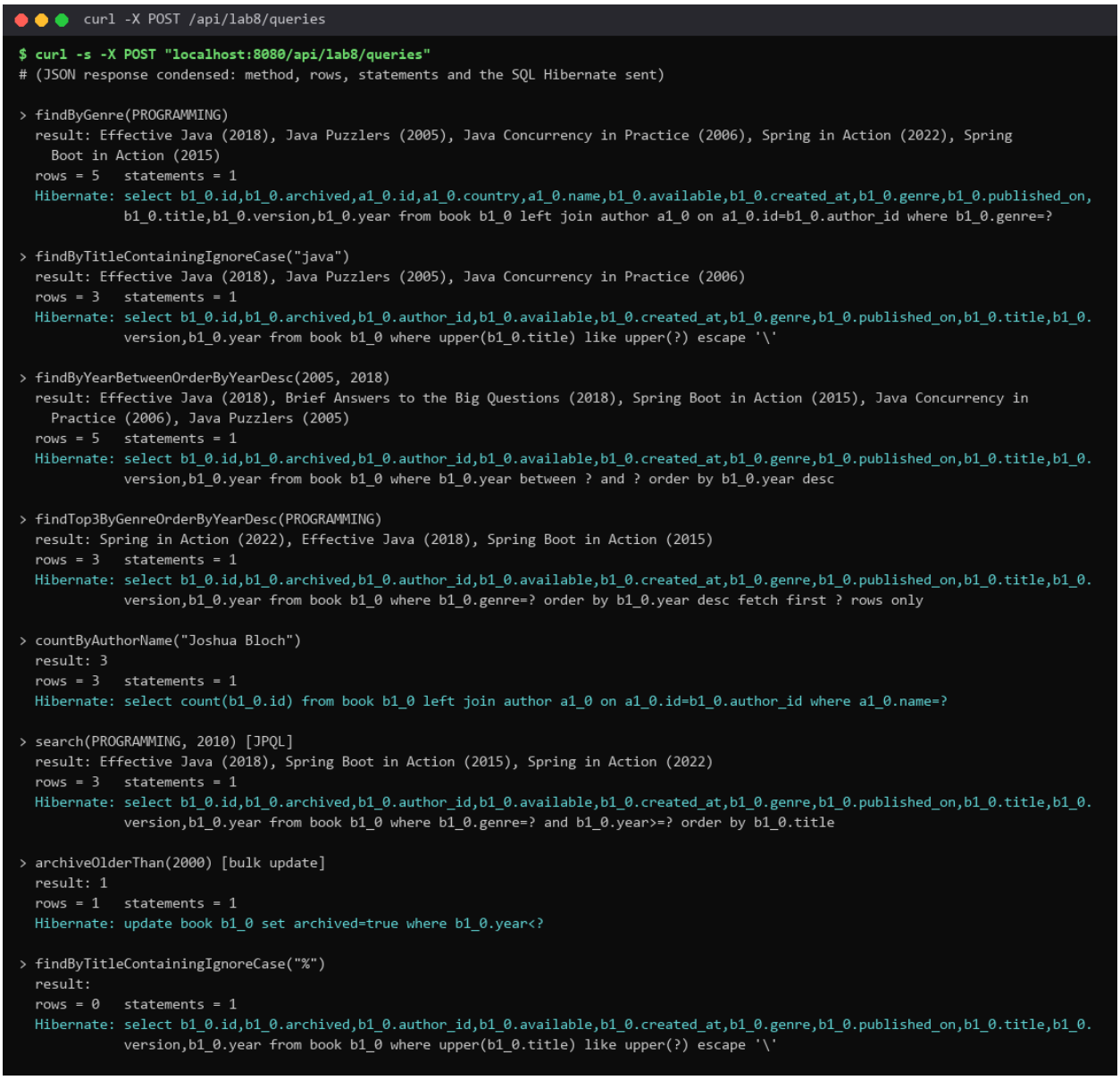


Figure 10. *POST /api/lab8/queries: results, statement counts and the SQL of every method*

Listing 8. *SQL that Hibernate generated for findTop3ByGenreOrderByYearDesc(PROGRAMMING)*

```
select b1_0.id, b1_0.archived, b1_0.author_id, b1_0.available, b1_0.created_at, b1_0.genre,
b1_0.published_on, b1_0.title, b1_0.version, b1_0.year
from book b1_0
where b1_0.genre=?
order by b1_0.year desc
fetch first ? rows only
```

Subject "findTop3" → fetch first 3 rows only; predicate "ByGenre" → where genre = ?; "OrderByYearDesc" → order by year desc. The JPQL that Spring Data builds is select b from Book b where b.genre = ?1 order by b.year desc, executed with maxResults = 3.

The bulk update archiveOlderThan(2000) is a single statement — update book set archived=true where year<? — and changed 1 row (A Brief History of Time, 1988). It needs @Modifying and a read-write transaction (BookService.archiveOlderThan is @Transactional).

3.4. The N+1 exercise

Table 4. *Task 3.4 — N+1 before and after*

Variant	Books	SQL statements
findAll() + book.getAuthor().getName()	8	4
findAllWithAuthor() [join fetch]	8	1
findByGenre(PROGRAMMING) [@EntityGraph]	5	1

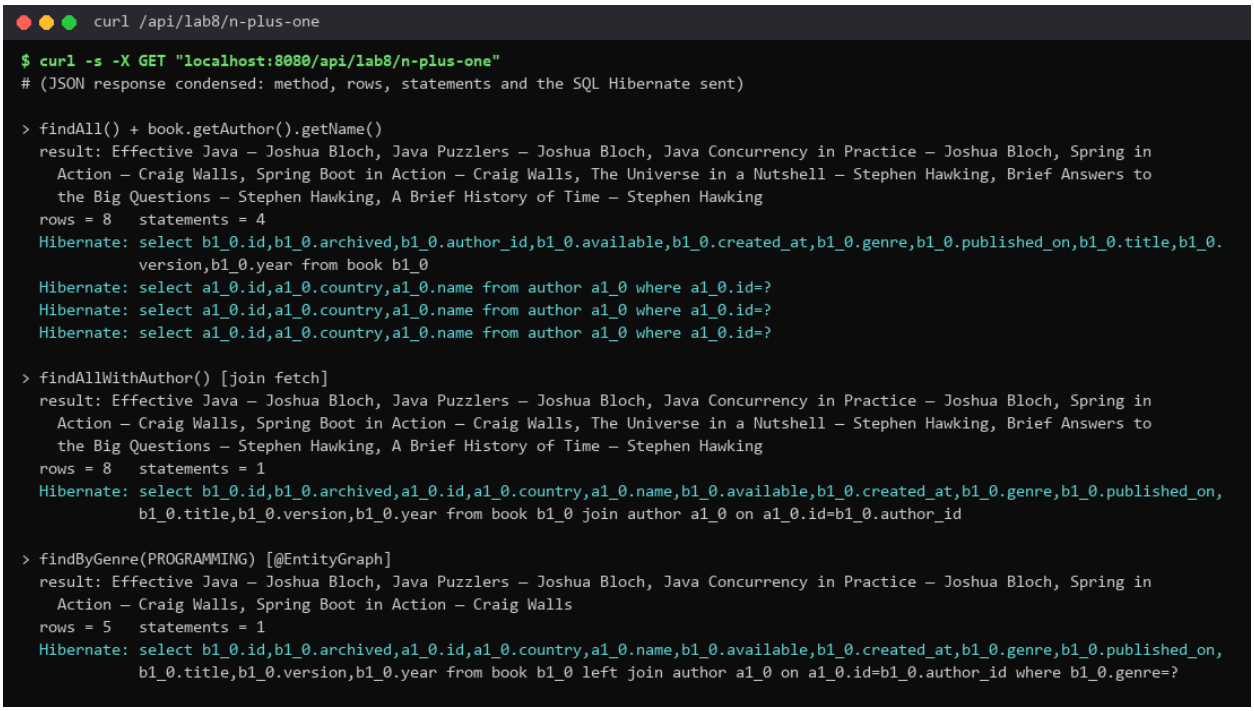


Figure 11. *GET /api/lab8/n-plus-one: 4 statements before the fetch join, 1 after (and 1 with @EntityGraph)*

Before the fetch join: 4 statements. After: 1. findAll() loads the 8 books with author_id only (author is LAZY). Each call book.getAuthor().getName() initialises a proxy with select ... from author where id=?. There are 3 distinct authors, and the persistence context returns an already loaded author from its first-level cache, so the count is 1 + 3 = 4, not 1 + 8; with 1 000 books by 300 authors it would be 301. select b from Book b join fetch b.author brings the author columns in the same row, so it is 1 statement; @EntityGraph(attributePaths = "author") on findByGenre does the same declaratively (with a left join).

5. Task 4. Pagination and sorting

Listing 9. *BookRestController — the collection endpoint is paged (@PageableDefault size 5, sort by title)*

```
    @GetMapping
    public Page<BookResponse> list(
            @Parameter(description = "Only books of this genre", example = "PROGRAMMING")
            @RequestParam(required = false) Genre genre,
            @Parameter(description = "Part of the author's name, case-insensitive", example = "Bloch")
            @RequestParam(required = false) String author,
            @ParameterObject @PageableDefault(size = 5, sort = "title") Pageable pageable) {
        return service.find(genre, author, pageable).map(mapper::toResponse);
    }
```

Listing 10. *BookService.find — the transaction boundary; the page brings its authors, the tags come in one batch*

```
    @Audited(action = "BOOK_LIST", logArguments = true)
    public Page<Book> find(Genre genre, String author, Pageable pageable) {
        boolean byAuthor = author != null && !author.isBlank();
        Page<Book> page;
        if (genre != null && byAuthor) {
            page = repository.findByGenreAndAuthorNameContainingIgnoreCase(genre, author.trim(),
pageable);
        } else if (genre != null) {
            page = repository.findByGenre(genre, pageable);
        } else if (byAuthor) {
            page = repository.findByAuthorNameContainingIgnoreCase(author.trim(), pageable);
        } else {
            page = repository.findAll(pageable);
        }
        page.forEach(book -> book.getTags().size());     // initialises the lazy collections
(@BatchSize)
        return page;
    }
```

PagingConfig enables @EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO), so the JSON shape is stable: content plus page {size, number, totalElements, totalPages}. spring.data.web.pageable.max-page-size = 50 prevents ?size=100000 from asking for the whole table.

4.2. The checks

curl — paged response

```
$ curl -s "localhost:8080/api/books?page=1&size=3&sort=year,desc"
{
    "content": [
        {
            "id": 5,
            "title": "Spring Boot in Action",
            "author": "Craig Walls",
            "year": 2015,
            "genre": "PROGRAMMING",
            "available": true,
            "archived": false,
            "publishedOn": "2015-12-01",
            "tags": [
                "spring"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        },
        {
            "id": 3,
            "title": "Java Concurrency in Practice",
            "author": "Joshua Bloch",
            "year": 2006,
            "genre": "PROGRAMMING",
            "available": false,
            "archived": false,
            "publishedOn": "2006-05-09",
            "tags": [
                "classic",
                "java"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        },
        {
            "id": 2,
            "title": "Java Puzzlers",
            "author": "Joshua Bloch",
            "year": 2005,
            "genre": "PROGRAMMING",
            "available": true,
            "archived": false,
            "publishedOn": "2005-07-01",
            "tags": [
                "java"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        }
    ],
    "page": {
        "size": 3,
        "number": 1,
        "totalElements": 8,
        "totalPages": 3
    }
}
```

Figure 12. *GET /api/books?page=1&size=3&sort=year,desc — the second page, newest first, totalElements = 8, totalPages = 3*

Listing 11. *The JSON of the paged response*

```
$ curl -s "localhost:8080/api/books?page=1&size=3&sort=year,desc"
{
    "content": [
        {
            "id": 5,
            "title": "Spring Boot in Action",
            "author": "Craig Walls",
            "year": 2015,
            "genre": "PROGRAMMING",
            "available": true,
            "archived": false,
            "publishedOn": "2015-12-01",
            "tags": [
                "spring"
```

```
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        },
        {
            "id": 3,
            "title": "Java Concurrency in Practice",
            "author": "Joshua Bloch",
            "year": 2006,
            "genre": "PROGRAMMING",
            "available": false,
            "archived": false,
            "publishedOn": "2006-05-09",
            "tags": [
                "classic",
                "java"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        },
        {
            "id": 2,
            "title": "Java Puzzlers",
            "author": "Joshua Bloch",
            "year": 2005,
            "genre": "PROGRAMMING",
            "available": true,
            "archived": false,
            "publishedOn": "2005-07-01",
            "tags": [
                "java"
            ],
            "createdAt": "2026-10-26T14:42:34.182596Z"
        }
    ],
    "page": {
        "size": 3,
        "number": 1,
        "totalElements": 8,
        "totalPages": 3
    }
}
```



Figure 13. *The SQL of that request: the page query (offset / fetch first), the count query, and the batched select of the tags*

The paged request caused (1) select ... from book left join author ... order by year desc offset ? rows fetch first ? rows only — the page itself; (2) select count(b1_0.id) from book — the count query. It is there because a Page promises totalElements and totalPages, which the page query alone cannot know. The third statement (select ... from book_tag join tag where book_id = any(?)) is not part of paging: it loads the tags of all three books at once (@BatchSize) instead of one select per book.

sort=nosuchfield



Figure 14. *Before: 500 Internal Server Error, PropertyReferenceException "No property nosuchfield found for type Book"*



Figure 15. *After the handler in ApiExceptionHandler: 400, application/problem+json, type unknown-sort-property*

The sort parameter is only a string until Spring Data resolves it against the entity while it builds the query — at run time, inside SimpleJpaRepository (QueryUtils.toOrders → PropertyPath.from). The compiler never sees it, so it can only fail at run time. The project now maps PropertyReferenceException to a 400 problem detail: it is the client who asked for a property that does not exist.

Listing 12. *ApiExceptionHandler — unknown sort property → 400*

```
    @ExceptionHandler(PropertyReferenceException.class)
    public ProblemDetail handleUnknownProperty(PropertyReferenceException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST,
ex.getMessage());
        problem.setType(URI.create(ERRORS + "unknown-sort-property"));
        problem.setTitle("Unknown sort property");
        problem.setProperty("property", ex.getPropertyName());
        return problem;
    }
```

4.3. Page against Slice

Table 5. *Task 4.3 — the same request (page 1, size 3, year desc)*

Return type	Rows	SQL statements	What the client learns
Page<Book> findAll(pageable)	3	2	content + totalElements = 8, totalPages = 3
Slice<Book> findSliceBy(pageable)	3	1	content + hasNext = true

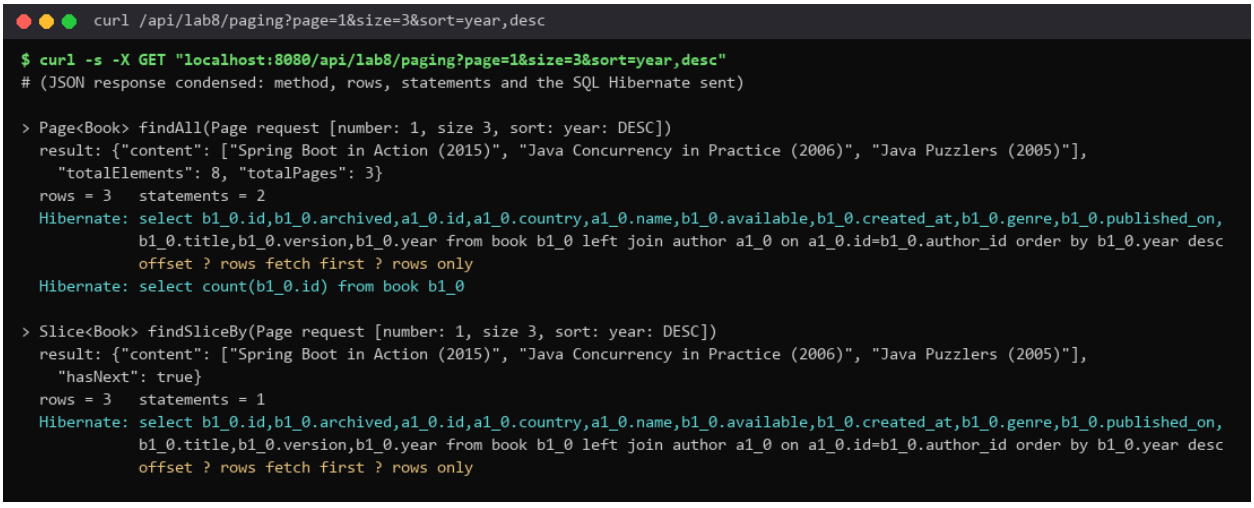


Figure 16. *GET /api/lab8/paging: a Page costs select + count, a Slice only the select*

The Slice asks for size + 1 rows (fetch first 4) and uses the extra row only to know whether there is a next page. Instead of changing the return type back and forth, the comparison uses a second repository method, findSliceBy, so both numbers come from the same run. What I would publish: a Page for this catalogue API — clients show "page 2 of 3", and the count over a small indexed table is cheap. For an infinite feed or a very large table I would publish a Slice (or keyset scrolling, see variant 7), because the count query can cost as much as reading the whole table.

4.4. Commit and push

```
PowerShell — library-portal (lab08)
$ git add .
$ git commit -m "lab08: PostgreSQL, entities, associations, queries, paging"
[lab08 dc290ed] lab08: PostgreSQL, entities, associations, queries, paging
 95 files changed, 3072 insertions(+), 138 deletions(-)
$ git push -u origin lab08
To https://github.com/AkAyu44/library-portal.git
 * [new branch]      lab08 -> lab08
branch 'lab08' set up to track 'origin/lab08'.
$ git log --oneline -3
dc290ed lab08: PostgreSQL, entities, associations, queries, paging
fe099ac docs: formal navy-and-gold defence deck with screenshots of the application
762481e docs: defence script without code — presentation text, tests, site walk-through
```

Figure 17. *git commit and git push -u origin lab08*

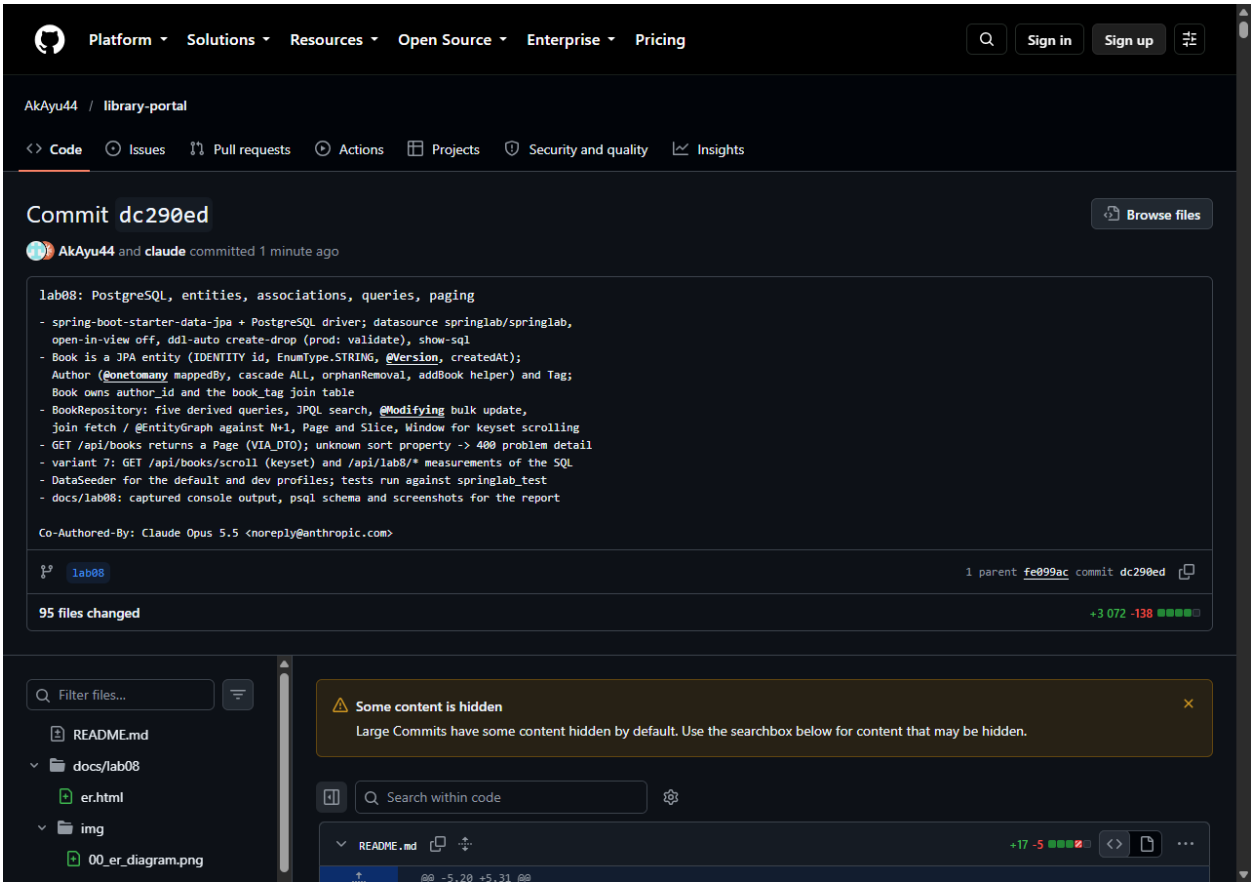


Figure 18. *The commit on GitHub, branch lab08*

6. What else the ORM did (used in the answers)

```
POST /api/books — INSERTs (author found, tags created)

----- SQL -----
Hibernate: select b1_0.id from book b1_0 left join author a1_0 on a1_0.id=b1_0.author_id where upper(b1_0.title)=upper(?) and
           upper(a1_0.name)=upper(?) fetch first ? rows only
Hibernate: select t1_0.id, t1_0.name from tag t1_0 where upper(t1_0.name)=upper(?)
Hibernate: select t1_0.id, t1_0.name from tag t1_0 where upper(t1_0.name)=upper(?)
Hibernate: select a1_0.id, a1_0.country, a1_0.name from author a1_0 where upper(a1_0.name)=upper(?)
Hibernate: insert into author (country, name) values (?, ?)
Hibernate: insert into book (archived, author_id, available, created_at, genre, published_on, title, version, year) values (?, ?,
           ?, ?, ?, ?, ?, ?, ?)
Hibernate: insert into book_tag (book_id, tag_id) values (?, ?)
Hibernate: insert into book_tag (book_id, tag_id) values (?, ?)
```

Figure 19. *POST /api/books: existing tags found, a new author inserted, the book inserted with author_id, one book_tag row per tag*

```
PUT /api/books/1 — dirty checking + @Version

----- SQL -----
Hibernate: select b1_0.id, b1_0.archived, a1_0.id, a1_0.country, a1_0.name, b1_0.available, b1_0.created_at, b1_0.genre,
           b1_0.published_on, t1_0.book_id, t1_1.id, t1_1.name, b1_0.title, b1_0.version, b1_0.year from book b1_0 left join
           author a1_0 on a1_0.id=b1_0.author_id left join book_tag t1_0 on b1_0.id=t1_0.book_id left join tag t1_1 on
           t1_1.id=t1_0.tag_id where b1_0.id=?
Hibernate: select b1_0.id from book b1_0 left join author a1_0 on a1_0.id=b1_0.author_id where upper(b1_0.title)=upper(?) and
           upper(a1_0.name)=upper(?) and b1_0.id<>? fetch first ? rows only
Hibernate: select t1_0.id, t1_0.name from tag t1_0 where upper(t1_0.name)=upper(?)
Hibernate: select t1_0.id, t1_0.name from tag t1_0 where upper(t1_0.name)=upper(?)
Hibernate: update book set archived=?, author_id=?, available=?, genre=?, published_on=?, title=?, version=?, year=? where id=?
           and version=?

$ psql -c "select id, title, version from book where id = 1;"
 id |           title            | version
----+----------------------------+---------
  1 | Effective Java, 3rd edition |       1
(1 row)
```

Figure 20. *PUT /api/books/1: no save() in the service — dirty checking issues the UPDATE "where id=? and version=?"; version became 1*

Listing 13. *BookService.update — changes a managed entity and never calls save()*

```
    @Transactional
    @Audited(action = "BOOK_UPDATE", logArguments = true)
    public Book update(long id, BookCommand command) {
        Book book = findById(id);           // throws BookNotFoundException
        rejectDuplicate(command, id);
        citations.evict(id);
        book.setTitle(command.title().trim());
        book.setGenre(command.genre());
        book.setYear(command.year());
        book.setAvailable(command.available());
        book.setPublishedOn(command.publishedOn());
        if (book.getAuthor() == null ||
!book.getAuthor().getName().equalsIgnoreCase(command.author().trim())) {
            author(command.author()).addBook(book);
        }
        book.clearTags();
        command.tags().forEach(name -> book.addTag(tag(name)));
        return book;
    }
```

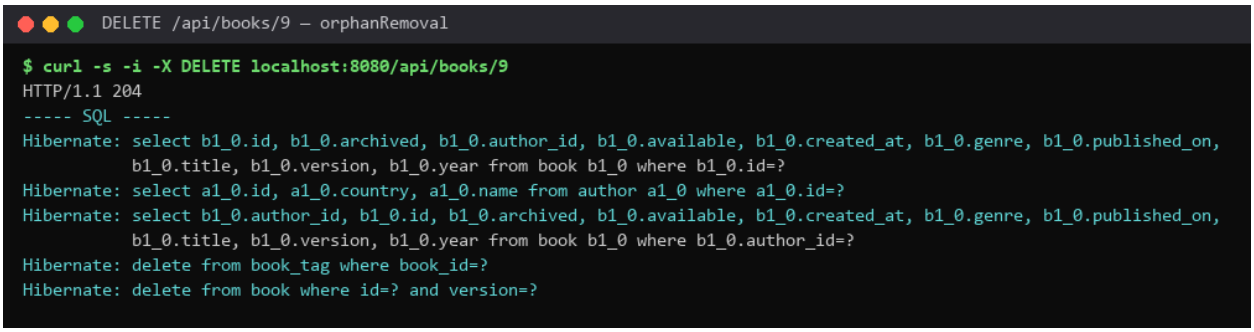


Figure 21. *DELETE /api/books/9: author.removeBook(book) → orphanRemoval deletes the book_tag rows and the book "where id=? and version=?"*

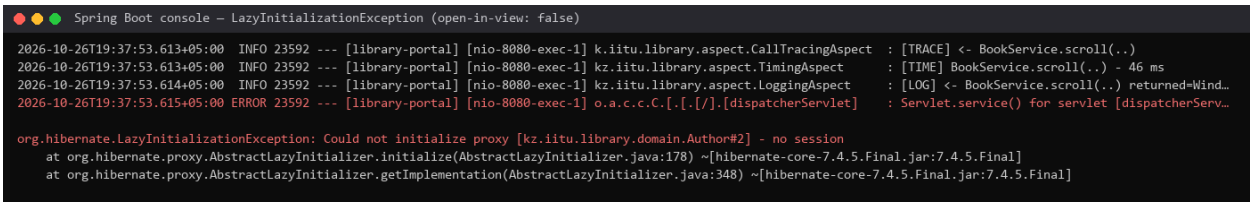


Figure 22. *LazyInitializationException met during the work: with open-in-view off, the controller touched a lazy author after the transaction had ended*

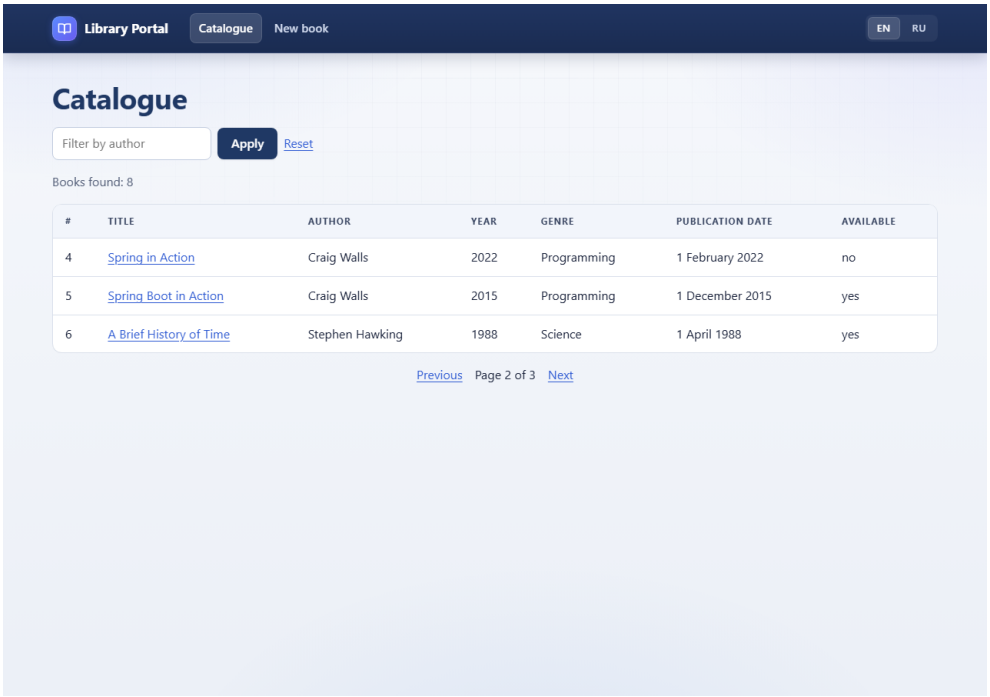


Figure 23. *The Thymeleaf catalogue is paged too: /books?size=3&page=1 — "Page 2 of 3"*

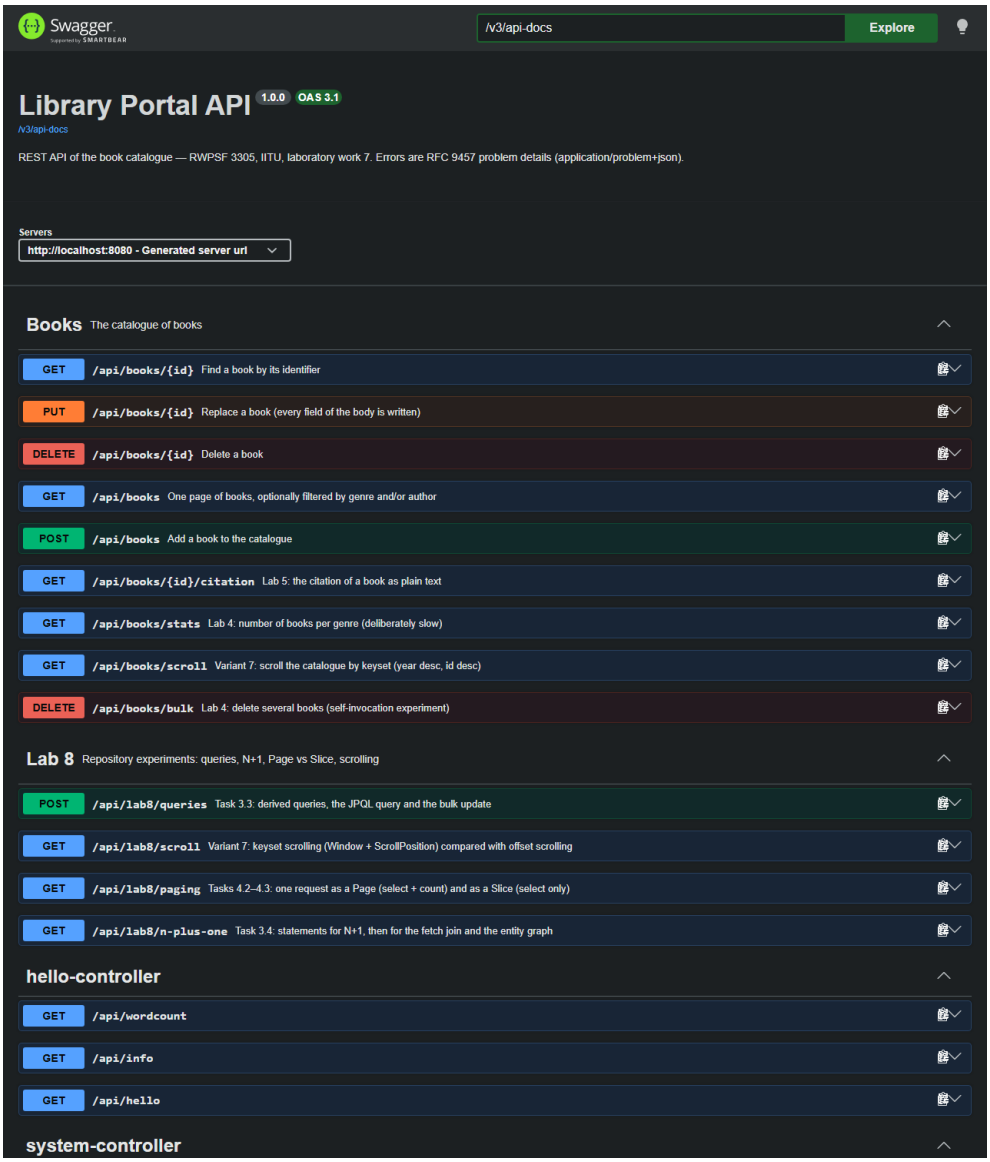


Figure 24. *Swagger UI: the paged GET /api/books, GET /api/books/scroll and the Lab 8 group*

7. Individual variant 7 — Window and keyset scrolling

Requirement: Window<Book> with a ScrollPosition; compare the SQL with offset paging.

Listing 14. *BookRepository — the Window method*

```
// ---------------- Individual variant 7: Window and keyset scrolling ----------------

/**
 * Keyset (seek) scrolling: the next window starts after the sort keys of the last row —
 * where (year < ? or year = ? and id < ?) — instead of skipping OFFSET rows.
 * BookService never passes ScrollPosition.keyset() (the empty initial position): Spring Data
 * caches the query rendered for the first call, and a query rendered without the keyset
 * predicate would then be reused for every later window (found in this lab, see the report).
 */
@EntityGraph(attributePaths = "author")
Window<Book> findBy(ScrollPosition position, Sort sort, Limit limit);
```

Listing 15. *BookService.scroll — newest first; the id is the tie-breaker of the sort keys*

```
/**
 * Lab 8, variant 7: the first window starts "after" a book newer than any real one.
 * Every call then has the same query shape (with the keyset predicate) — see BookRepository.findBy.
 */
public static final ScrollPosition FIRST_WINDOW =
        ScrollPosition.forward(Map.of("year", Integer.MAX_VALUE, "id", Long.MAX_VALUE));

public Window<Book> scroll(ScrollPosition position, int size) {
    if (position.isInitial()) {
        position = FIRST_WINDOW;
    }
    Window<Book> window = repository.findBy(position, NEWEST_FIRST, Limit.of(size));
    window.forEach(book -> book.getTags().size());
    return window;
}
```

Listing 16. *BookRestController.scroll — GET /api/books/scroll?size=3&afterYear=&afterId=*

```
@GetMapping("/scroll")
public BookWindowResponse scroll(
        @Parameter(description = "Books per window", example = "3")
        @RequestParam(defaultValue = "3") int size,
        @Parameter(description = "Year of the last book of the previous window", example = "2018")
        @RequestParam(required = false) Integer afterYear,
        @Parameter(description = "Id of the last book of the previous window", example = "8")
        @RequestParam(required = false) Long afterId) {
    ScrollPosition position = afterYear == null || afterId == null
            ? ScrollPosition.keyset()
            : ScrollPosition.forward(Map.of("year", afterYear, "id", afterId));
    Window<Book> window = service.scroll(position, Math.clamp(size, 1, 50));
    List<BookResponse> content = mapper.toResponseList(window.getContent());
    BookWindowResponse.Cursor next = null;
    if (window.hasNext() && !content.isEmpty()) {
        BookResponse last = content.getLast();
        next = new BookWindowResponse.Cursor(last.year(), last.id());
    }
    return new BookWindowResponse(content, window.hasNext(), next);
}
```

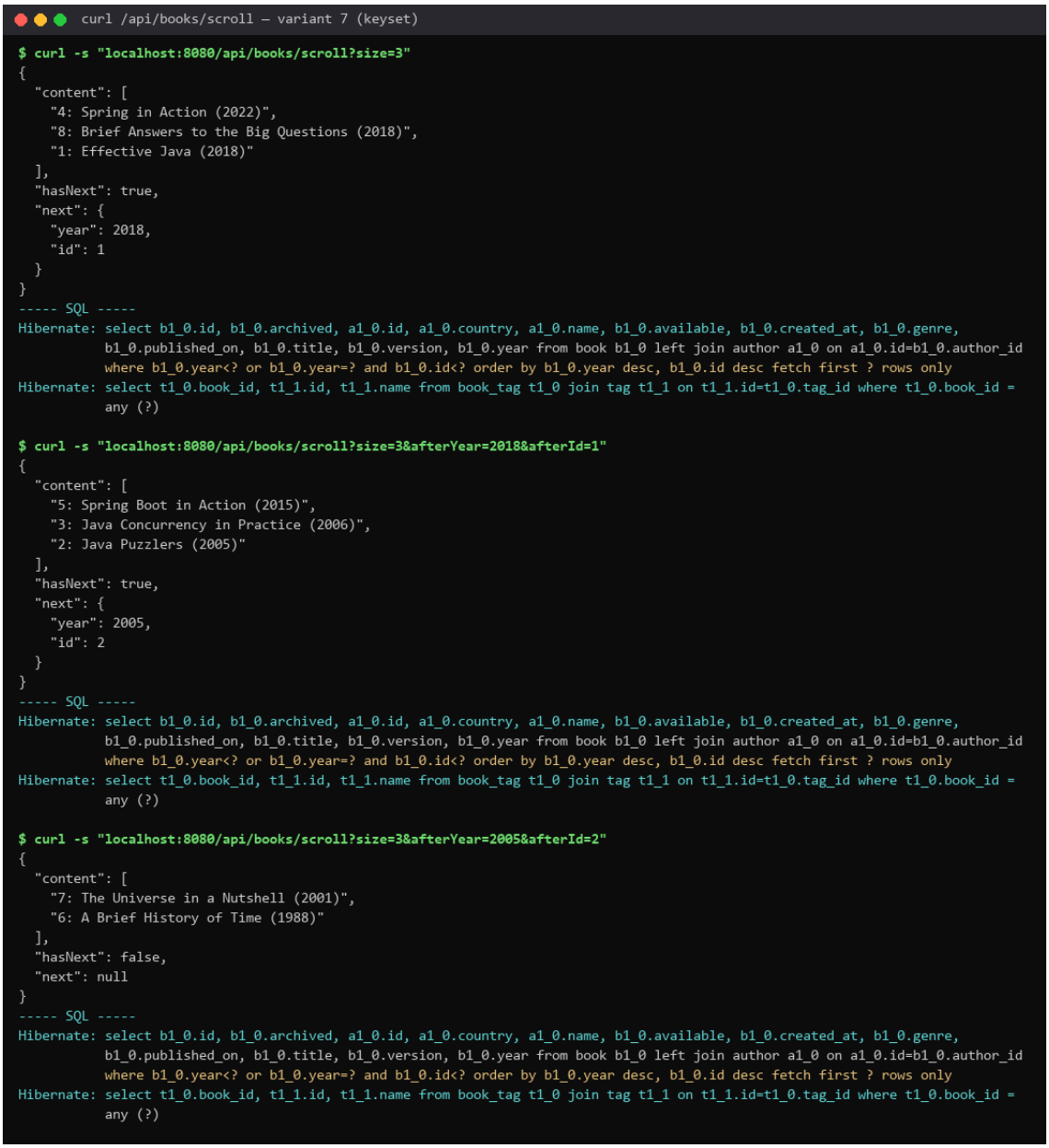


Figure 25. *Three windows of GET /api/books/scroll: each "next" cursor is the (year, id) of the last book; the third window has hasNext = false*

Table 6. *Variant 7 — keyset against offset (GET /api/lab8/scroll)*

Request	Books	SQL	Key part of the SQL
Keyset, first window	3	1	where year<? or year=? and id<? order by year desc, id desc fetch first ? rows only
Keyset, after (2018, id 1)	3	1	where year<? or year=? and id<? ... fetch first ? rows only
Offset Page, page 1, size 3	3	2	order by year desc, id desc offset ? rows fetch first ? rows only + select count(...)

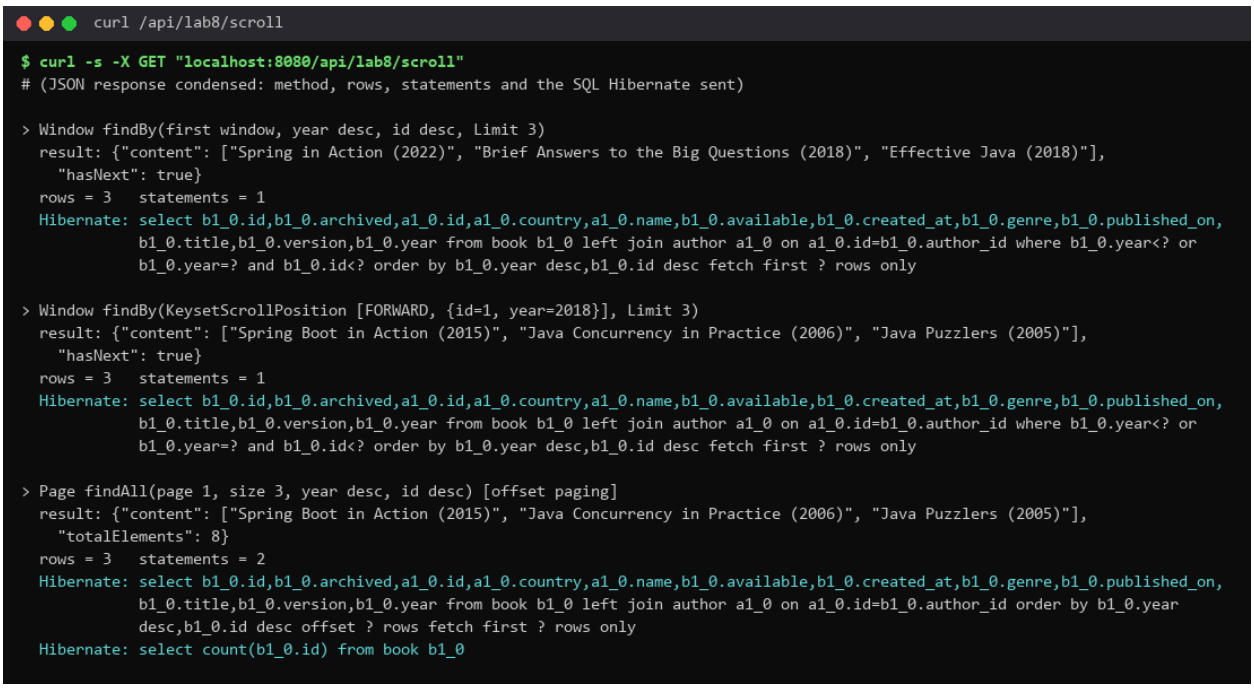


Figure 26. *GET /api/lab8/scroll: the same second "page" by keyset and by offset*

Comparison. Offset paging makes the database produce the first page × size rows, throw them away and return the next size rows: the deeper the page, the more rows are read, and a row inserted or deleted between two requests shifts the pages (a book is shown twice or skipped). The Page version also adds a count query. Keyset scrolling remembers the sort keys of the last row and asks only for the rows "after" them: with an index on (year, id) the database seeks straight to the position, the cost does not grow with depth, concurrent inserts do not shift the window, and there is no count. The price: no "jump to page 7" and no total — only "next".

A pitfall found while doing the variant. The first version passed ScrollPosition.keyset() (the empty initial position) for the first window. The second window then returned the same three books, and its SQL had no where clause at all. After a restart, asking for a window with a cursor first produced the correct "where year<? or year=? and id<?" — so Spring Data JPA 4.1.1 caches the query it renders for the first call of the method, and the query rendered for the empty position (without the keyset predicate) was reused for every later position. The fix: never pass the empty position — the first window starts "after" a sentinel (Integer.MAX_VALUE, Long.MAX_VALUE), so every call has the same query shape. The test Lab8RepositoryTests.keysetScrolling walks the whole catalogue window by window and asserts that no book repeats and that no statement contains OFFSET.

8. Automated checks

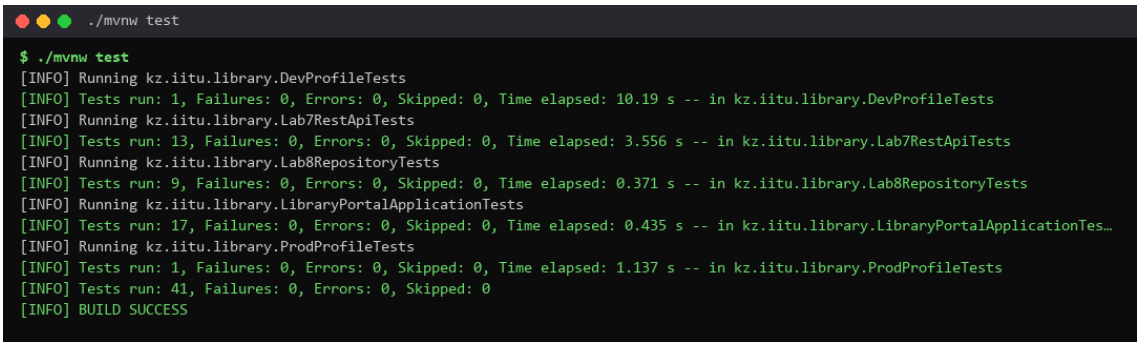


Figure 27. *./mvnw test — 41 tests, 0 failures (9 new ones in Lab8RepositoryTests), run against the PostgreSQL database springlab_test*

Test (Lab8RepositoryTests)	What it proves
derivedQueries	the five derived queries return rows from the seed
percentIsEscaped	findByTitleContainingIgnoreCase("%") matches nothing — the % is escaped
jpqlAndBulkUpdate	JPQL search with named parameters; the bulk update archives 1 row; clearAutomatically makes the next read see it
owningSideWritesTheForeignKey	after create(...) the row in book has the author_id of Joshua Bloch and two book_tag rows
nPlusOne	findAll + getAuthor = 1 + number of authors statements; findAllWithAuthor = 1
pageAndSlice	Page = 2 statements (one is select count), Slice = 1
keysetScrolling	windows cover the catalogue without repeats, no OFFSET in the SQL
pagedEndpoint	GET /api/books?page=1&size=3&sort=year,desc → 3 items, number 1, the total
unknownSortPropertyIs400	sort=nosuchfield → 400 problem detail

9. Demonstration checklist (section 7 of the handout)

No.	What is shown	Where (report / application)
1	The application starting against PostgreSQL	figure 2: jdbc:postgresql, PostgreSQL 16.14; BookRepository extends JpaRepository
2	The Book entity	Listing 3; @Id, @GeneratedValue(IDENTITY), protected constructor; answers 1–2
3	The tables in the database	figure 3, figure 4, figure 9
4	The two associations and their owning sides	Listings 4–5, figure 7; answer 5
5	The author_id column and the book_tag table	Table 2, figure 9
6	The addBook helper	Listing 5; answer 8
7	Three derived queries running, SQL read aloud	POST /api/lab8/queries — table 3, figure 10, listing 8
8	The JPQL query and the bulk update	search(:genre, :year), archiveOlderThan with @Modifying; answer 15
9	The N+1 count, before and after	Table 4, figure 11: 4 → 1
10	GET /api/books?page=1&size=3&sort=year,desc	figure 12: 3 books, year desc, total 8
11	The SQL of the paged query	figure 13: the page query and select count
12	The individual variant in action	Section 7, figure 25, table 6

How to run the demonstration: start PostgreSQL (pg_ctl -D C:\Users\kuatw\tools\pgdata start), run ./mvnw spring-boot:run, open psql -U springlab -d springlab (\d book, \d book_tag), and call /api/books?page=1&size=3&sort=year,desc, POST /api/lab8/queries, /api/lab8/n-plus-one, /api/lab8/paging, /api/lab8/scroll and /api/books/scroll; the console shows the SQL (show-sql + format_sql).

10. Answers to the questions of the demonstration (section 10 of the handout)

1. Why can your Book no longer be a record? What exactly does JPA require of the class?

JPA requires: @Entity; a primary key marked with @Id; a public or protected no-argument constructor; a top-level class that is not final, with persistent fields that are not final. Hibernate instantiates the entity with the no-arg constructor and then sets the fields from the row; it creates proxy subclasses for lazy references; it changes the fields of managed entities and compares them at flush. A record is implicitly final, its fields are final and it has only the canonical constructor, so none of this is possible and the application would fail at start-up. Book is now a class with a protected constructor for JPA and a public Book(title, genre, year) for the code.

2. What does @GeneratedValue(strategy = IDENTITY) do, and where does the value come from?

The value is generated by the database: the column is created as "id bigint generated by default as identity" (figure 4) and PostgreSQL takes the next value of the identity sequence on INSERT. Hibernate does not know the id before the insert, so persist()/save() runs the INSERT immediately and reads the generated key back (that is why IDENTITY inserts cannot be batched). Until then the id is null.

3. Why is your enum mapped with EnumType.STRING? What would break with ORDINAL?

STRING stores the name: genre varchar(20) with values like PROGRAMMING, and Hibernate even adds a check constraint genre in (NOVEL, SCIENCE, ...). ORDINAL stores the position (0, 1, 2 ...). If a constant is inserted in the middle or the enum is reordered, every existing row silently changes its meaning (2 was PROGRAMMING, now it is HISTORY), and the data cannot be read without the Java source. With STRING only a rename is dangerous, and it fails loudly.

4. What is the @Version field for? What happens when two transactions change the same row?

It is optimistic locking. Every UPDATE and DELETE is written as "... where id=? and version=?" and increments the version (figure 20: version 0 → 1). If two transactions both read version 0 and both update, the first commits and makes it 1; the second one's "where version=0" matches 0 rows, Hibernate throws OptimisticLockException (in Spring: ObjectOptimisticLockingFailureException) and that transaction is rolled back. The lost update is detected instead of silently overwriting the first change; the client reloads and retries (an API would answer 409 Conflict).

5. Which side of the book–author association owns it? How do you know from the code alone?

Book.author owns it: it is the @ManyToOne side with @JoinColumn(name = "author_id"), the side that maps the column. Author.books has mappedBy = "author", which literally says "this association is mapped by the field author of Book": an inverse side has mappedBy and no column. In a @ManyToOne/@OneToMany pair the @ManyToOne side always owns, because the foreign key lives in the table of the "many".

6. What does mappedBy mean? What would happen if you removed it from Author.books?

mappedBy names the field of the other entity that owns the association, so this collection is only a mirror and JPA ignores it when writing. Without mappedBy JPA would see two independent associations: Book.author with author_id, and a unidirectional @OneToMany Author.books, which by default is mapped with a join table — Hibernate would create a third table author_books(author_id, books_id) and store every link twice, in two places that can disagree.

7. Which table holds the many-to-many? What are its columns, and who created it?

book_tag (figure 9): book_id bigint not null → book(id), tag_id bigint not null → tag(id), primary key (book_id, tag_id). Hibernate created it (ddl-auto: create-drop) from @JoinTable(name = "book_tag", joinColumns = book_id, inverseJoinColumns = tag_id) on Book.tags; there is no entity class for it.

8. Why does addBook set both sides? Show what the SQL does if it sets only one.

Because the Java objects and the database are two different worlds. The database needs book.author_id, and JPA reads it only from the owning side, Book.author. The objects need author.books to contain the book, or the same transaction sees an inconsistent graph. If only author.getBooks().add(book) is called, the insert is "insert into book (..., author_id, ...) values (..., null, ...)" and the link is lost. If only book.setAuthor(author) is called, the database is right but author.getBooks() in the same persistence context does not contain the book. addBook does both; in the project setAuthor is package-private, so the rest of the code cannot skip the helper.

9. What does CascadeType.ALL on Author.books cause when an author is deleted?

Every operation on the author is repeated on its books: persist (saving an author saves its new books — the seed saves only authors), merge, refresh, detach and remove. Deleting an author therefore first deletes all its books (and their book_tag rows) and then the author; without the cascade the delete would fail on the foreign key book.author_id.

10. What is orphanRemoval = true doing that the cascade does not?

A cascade reacts to an operation on the parent. orphanRemoval reacts to a child being taken out of the parent's collection while the parent stays: a book removed from author.books is an orphan and is deleted. In the project DELETE /api/books/9 calls author.removeBook(book) and never calls repository.delete; figure 21 shows the resulting "delete from book_tag where book_id=?" and "delete from book where id=? and version=?".

11. Where is the implementation of BookRepository? Who wrote it and when?

Nobody wrote it by hand. At start-up Spring Data JPA (JpaRepositoryFactory) creates a JDK dynamic proxy for the interface (jdk.proxy4.$Proxy188 in the stack trace of figure 14). Calls to the CRUD and paging methods go to the generic SimpleJpaRepository; for every other method Spring Data parses the name into a PartTree once, builds the query and executes it through the EntityManager; @Query methods use the given JPQL.

12. Split findTop3ByGenreOrderByYearDesc into subject and predicate and read out the JPQL.

Subject: "find" (a query) + "Top3" (limit 3). "By" separates the subject from the predicate. Predicate: "Genre" (genre = ?1). Order clause: "OrderBy" "Year" "Desc". JPQL: select b from Book b where b.genre = ?1 order by b.year desc, with maxResults 3. SQL: select ... from book b1_0 where b1_0.genre=? order by b1_0.year desc fetch first ? rows only (listing 8).

13. What SQL does findByTitleContainingIgnoreCase produce? What happens to a % in the argument?

select ... from book b1_0 where upper(b1_0.title) like upper(?) escape '\'. Spring Data wraps the argument as %argument% and escapes the special characters % and _ inside it with the escape character \, so a % in the argument is a literal percent sign, not a wildcard: findByTitleContainingIgnoreCase("%") returned 0 rows (table 3), while an unescaped "%%%" would have matched all 8 books.

14. Why did you use named parameters in the JPQL query rather than ?1?

:genre and :year are bound by name with @Param, so the query reads by itself, the order of the method parameters can change without breaking it, the same parameter can be used twice, and a mismatch is reported

at start-up. ?1/?2 depend on positions and break silently when someone reorders the arguments. Both are bind parameters — neither is string concatenation — so both are safe from SQL injection.

15. Why does the bulk update need @Modifying? What does clearAutomatically = true prevent?

Spring Data runs an @Query with getResultList()/getSingleResult() by default; an UPDATE must be run with executeUpdate(), and @Modifying says so (without it: "Not supported for DML operations"). It also needs a transaction (else TransactionRequiredException) — BookService.archiveOlderThan is @Transactional. A bulk update goes straight to the database and bypasses the persistence context, so entities already loaded keep the old archived = false. clearAutomatically = true clears the context after the update, so the next read loads the fresh row; the test jpqlAndBulkUpdate checks exactly that.

16. Show the N+1 numbers. Explain where the extra queries came from and why the fetch join removes them.

Before: 4 statements, after: 1 (table 4, figure 11). findAll() selected the 8 books with author_id only, because author is LAZY. Touching book.getAuthor().getName() initialised the proxy with one select per author; there are 3 authors and the first-level cache serves the repeats, hence 1 + 3. "join fetch b.author" makes Hibernate select the author columns in the same statement and fill the association from the join, so no proxy is left to initialise.

17. What is the difference between join and join fetch in JPQL?

join only joins for filtering or ordering (where a.name = ...): the selected Book still has an uninitialised lazy author, and touching it costs a select. join fetch also selects the columns of the joined entity and initialises the association in the result, so the data comes in one statement. Fetching a collection together with paging makes Hibernate page in memory, which is why the page query fetches only the author and loads the tags with @BatchSize.

18. How many SQL statements does a Page cause? And a Slice? Why the difference?

Page: 2 — the select with offset/fetch first and select count(...) (table 5). Slice: 1 — it fetches size + 1 rows and uses the extra row only to set hasNext. A Page must report totalElements and totalPages, which only a count can give; a Slice promises only "is there a next one". (Spring Data skips the count when the first page is not full.)

19. What happened when you passed sort=nosuchfield? Why is that a run-time error?

At first: 500 Internal Server Error with PropertyReferenceException "No property nosuchfield found for type Book" (figure 14). The sort parameter is a string in the URL; Spring MVC puts it into the Pageable without knowing the entity, and only Spring Data resolves it against Book while building the query, inside the repository call. Nothing is compiled from it, so it cannot fail earlier. The project now answers 400 with a problem detail (figure 15).

20. What is ddl-auto set to in your project? What is the default on PostgreSQL, and what should it be in production?

create-drop in application.yml (this class, the default and dev runs), create in the tests (several test contexts share springlab_test), validate in application-prod.yml. Spring Boot's default is create-drop only for an embedded database (H2, HSQLDB, Derby); for PostgreSQL it is none — the "relation book does not exist" trap. In production: none or validate, with the schema created and changed by versioned migrations (Flyway or Liquibase); Hibernate must never drop or alter production tables.

21. Why did you set open-in-view: false? What would you have to do differently with it on?

With open-in-view on (Spring Boot's default, which logs a warning) the EntityManager stays open for the whole HTTP request, so lazy associations load silently during JSON serialisation or Thymeleaf rendering — hidden N+1 queries outside the service, and a database connection held while the view renders. With it off, the service transaction is the boundary: whatever the response needs must be fetched inside it (entity graph, join fetch, @BatchSize + initialisation in BookService.find); otherwise — LazyInitializationException (figure 22). With it on those fetches would not be needed for correctness, but I would have to watch the SQL log for the hidden queries.

11. Conclusions

The ORM took over everything between the objects and the tables: Hibernate created the four tables, the foreign keys and the join table from annotations, wrote the INSERTs and UPDATEs itself (dirty checking, the version check, orphan removal), and Spring Data turned method names into SQL — no SQL was written by hand except two JPQL queries. The REST API of lab 7 became paged and sorted with one Pageable parameter, and the individual variant added keyset scrolling, which reads only the rows it returns and needs no count.

What it took away is direct control and visibility: the number of statements depends on fetch types and on what the code touches (4 statements instead of 1 for N+1), a Page silently adds a count query, a missing transaction with open-in-view off turns into LazyInitializationException, a wrong sort property is found only at run time, and even the framework can surprise (the cached keyset query). Reading — and measuring — the SQL that Hibernate prints is therefore part of using an ORM, not an optional extra.